

למידת חיזוקים

בלמידת חיזוקים (Reinforcement Learning) הפעולות והתגמולים הם סטוכסטיים, ובניגוד ל-MDP רגיל אנחנו לא יודעים את ההסתברויות ואנחנו צריכים ללמוד אותן באמצעות הפעולות שאנחנו מבצעים.

למשל - יש "מבוך", ובכל משבצת יש הפעלה של פעולה. הפעולות יכולות להיות הסתברויות - כלומר אמרנו למשל לרובוט לזוז למעלה ויש הסתברות שהוא יזוז ימינה. אנחנו רוצים למצוא פוליסה אופטימלית.

באלגוריתם מונטה-קרלו, פשוט מפעילים פוליסה הרבה פעמים ובודקים את התוחלת של התגמול על הפוליסה.

באלגוריתם Adaptive Dynamic Programming, מפעילים פוליסה בשביל למדוד את ההסתברויות של תוצאות הפעולות (transition model) ושל פונקציית התגמול, ואז מחשבים את הערך האופטימלי לפי נוסחת בלמן:

$$V_{\pi}(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) \cdot V_{\pi}(s')$$

את ה transition model $P(s'|s, \pi(s))$ משערים לפי מספר הפעמים שראינו את s' אחרי שביצענו פעולה a במצב s .

לחשב את כל ההתפלגות הזאת זה מאוד יקר, ולכן במקום זה אפשר לעשות Temporal Difference Learning ולנסות להתכנס ל- V_{π} האופטימלי באמצעות:

$$V_{\pi}(s) := V_{\pi}(s) + \alpha (R(s) + \gamma V_{\pi}(s') - V_{\pi}(s))$$

α נקרא learning rate. ככל ש- α יותר קרוב ל-1 ככה אנחנו מתעדפים את הערכים החדשים, וככל ש- α יותר קרוב ל-0 ככה אנחנו מתעדפים את הערכים הישנים. איך בוחרים איזה פעולות לעשות?

- גישה נאיבית - פועלים אקראית במשך הרבה זמן, ואוספים מידע. הבעיה - זה לוקח המון זמן.

הערה: אם יש deadends יכול להיות שנגיע לאחד ומשך לא נוכל ללמוד.

- שיפור לגישה הנאיבית: מתחילים עם מודל לא מיועד, מוצאים פוליסה לפי המודל הזה, ואז בכל פעם מבצעים פעולה לפי הפוליסה ומתקנים את המודל והפוליסה לפי המידע החדש שקיבלנו.

ב-RL יש שתי סיבות לבצע פעולה:

- Exploitation - משתמשים בידע שלנו כדי לבצע פעולה כדי לקבל ערך.
- Exploration - מקבלים עוד מידע על העולם.

בשביל exploration בד"כ נאלץ לבצע פעולות שלא ממקסמות את הערך לפי מודל העולם הידוע הנוכחי. יש כאן tradeoff. לפי זה, נעשה ADP-based RL:

- נתחיל עם מודל התחלתי, נמצא פוליסה לפי המודל, ונבצע פעולות לפי פוליסת explore/exploit.

בשביל זה נשתמש ב-Q-function:

$$Q(s, a) = R(s) + \gamma \sum_{s'} P(s'|s, a) \cdot V_\pi(s')$$

$S(s, a)$ זה בעצם פונקציית ערך ל"כמה טוב יהיה לבצע פעולה a במצב s ".
פעולה חמדנית (Greedy Action) היא פעולה שממקסמת את Q-value:

$$Q(s, a) = R(s) + \gamma \sum_{s'} P(s'|s, a) \cdot V(s')$$

כאשר V הוא הערך האופטימלי לפי המודל הנוכחי.
נרצה פוליסה שהיא GLIE - greedy in the limit of infinite exploration.
• GLIE Policy 1: בשלב t נבחר פעולה אקראית בהסתברות $p(t)$ ופעולה חמדנית בהסתברות $1 - p(t) = \frac{1}{t}$. למשל, תכנס אותנו ל-greedy אבל זה יהיה מאוד איטי.

• GLIE Policy 2: Boltzmann Exploration. נבחר פעולה לפי:

$$P(a|s) = \frac{\exp\left(\frac{Q(s,a)}{T}\right)}{\sum_{a' \in A} \exp\left(\frac{Q(s,a')}{T}\right)}$$

כאשר T הוא ה"טמפרטורה" - ערך גבוה T אומר שלפעולות יש סיכוי אחיד, ו T נמוך אומר שפועלים יותר לפי greedy. מתחילים עם T גבוה ומקטינים עם הזמן.

גישה: Optimistic Exploration

מתחילים עם מודל התחלתי, ומוצאים פוליסה אופטימלית כרגיל, אבל מנפחים את הערכים של פעולות שעוד לא ראינו.

אנחנו רוצים בעצם לקבוע פונקציית ערך:

$$V^+(s) := R(s) + \gamma \cdot \max_a \begin{cases} \sum_{s'} P(s'|s, a) \cdot V^+(s') & \text{Explored} \\ V^{\max} & \text{Unexplored} \end{cases}$$

כאשר

$$V^{\max} = \sum_{t=0}^{\infty} \gamma^t \cdot R^{\max} = \frac{R^{\max}}{1 - \gamma}$$

גישה: TD-based Active RL

מתחילים עם מודל התחלתי, ומבצעים explore/exploit policy. מעדכנים לפי מעבר ממצב נצפה למצב s' - מבצעים עדכון TD:

$$V(s) := V(s) + \alpha (R(s) + \gamma V(s') - V(s))$$

הדבר הזה אינו model-free כי צריך ללמוד את ערכי V . האם אפשר ללכת ישירות על למידת ערכי Q ?

ניזכר ש $V(s) = \max_a Q(s, a)$ ונציב במשוואת בלמן:

$$Q(s, a) = R(s) + \gamma \sum_{s'} P(s'|s, a) Q(s', a) = R(s) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a')$$

אבל גם זה לא model-free כי זה דורש את ערכי P . לכן נחליף את זה ב:

$$Q(s, a) = R(s) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a')$$

הכללה באמצעות פיצ'רים

מרחב המצבים יכול להיות מאוד גדול. במקום ללמוד אותו, נרצה להכליל אותו באמצעות פיצ'רים. פיצ'ר הוא בעצם פונקציית ממרחב המצבים למרחב יותר קטן - בד"כ בינארי, אבל יכול גם להיות מספר.

נניח יש לנו n פיצ'רים $f_1(s), \dots, f_n(s)$ נתונים (כלומר מישוהו כבר הינדס אותם). נרצה ללמוד את המשקלות θ שנותנות לנו לחשב ערך $V(s)$ בצורה לינארית לפי הפיצ'רים:

$$V_\theta(s) = \theta_0 + \theta_1 f_1(s) + \dots + \theta_n f_n(s)$$

אם אי אפשר ללמוד משקלים לינארים לפיצ'רים, אפשר לפעמים להוסיף עוד פיצ'רים. הרעיון: לתת הרבה פיצ'רים ולראות מי מהם נופל בדרך - כלומר איזה משקלים מתאפסים.

אלגוריתם: TD-based RL for Linear Approximators

מתחילים עם פרמטרים התחלתיים, מבצעים פוליסת explore/exploit, מעדכנים את המודל, ומבצעים עדכון TD למשקלים θ_i .

מה זה עדכון TD לפרמטר? משתמשים ב Gradient Descent בשביל למזער את הטעות בין פונקציית השיערוך לערך האמיתי:

$$E_j = \frac{1}{2} (V_\theta(s_j) - v(s_j))^2$$

כאשר:	E_j	ריבוע השגיאה של דוגמה j
	$V_\theta(s_j)$	שערוך הערך עבור s_j
	$v(s_j)$	target value של s_j

כשרואים s_j מפעילים את gradient descent rule:

$$\theta_i := \theta_i - \alpha \frac{\partial E_j}{\partial \theta_i}$$

כלומר

$$\theta_i := \theta_i + \alpha (v(s_j) - V_\theta(s_j)) f_i(s_j)$$

בינה מלאכותית
89-5570-06

מקליד: עידן אריה
מרצה: ד"ר אלכסנדר שלייפמן
תאריך: 2026-05-28

בעיה: אנחנו לא באמת יודעים מה ה- $v(s_j)$.

נצטרך לשערך אותו לפי הדגימות:

$$\theta_i := \theta_i + \alpha (R(s) + \gamma V_\theta(s') - V_\theta(s_j)) f_i(s_j)$$

נשים לב: משתמשים בכל פעם רק ב- $R(s)$ כי לא שומרים ממוצעים על המצבים. המטרה היא לפרק לפיצ'רים.